AW AIOT Linux debian
开发指南

版本号: 1.0
发布日期: 2024.11.18

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2024.11.18	AWA1615	初始版本

广州星翼电子科技有限公司zdyzlinux

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1

2 AIOT Linux debian 介绍 **2**
2.1 debian 介绍 2
2.2 版本支持 2

3 debian 快速入门 **3**
3.1 目录结构 3
3.2 配置 3
3.3 构建 4
3.4 自定义 debian 基础包 4

4 debian 开发 **5**
4.1 配置处理 5
4.1.1 配置流程 5
4.1.2 快速配置 5
4.2 构建处理 6
4.2.1 构建流程 6
4.2.2 快速构建 6
4.3 debian 基础包编译 6
4.3.1 环境搭建 7
4.3.2 编译 7
4.3.3 软件源修改 8
4.3.4 rootfs 软件包修改 8
4.4 overlay 层叠加 8

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 前言

1.1 文档简介

本文档作为 Allwinner Debian Linux 系统软件开发指南，旨在帮助软件工程师、技术支持工程师快速上手，熟悉 AIOT Linux 软件开发及调试。

1.2 目标读者

本开发指南适用于 AIOT Linux Debian 软件开发工程师、AIOT 技术支持工程师。

1.3 适用范围

AIOT Linux v1.3 及以上版本。

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 AIOT Linux debian 介绍

2.1 debian 介绍

Debian 是一个自由、开源的 Linux 操作系统。

官方网站：https://www.debian.org/

2.2 版本支持

AIOT Linux SDK 当前仅支持 debian11(代号 bullseye)；暂未支持 debian12(代号 bookworm)；

桌面版本支持：

- xfce 桌面版本
- lxde 桌面版本
- gnome 桌面版本
- lite 无桌面版本

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 debian 快速入门

3.1 目录结构

AIOT Linux debian 主要用到 SDK 根目录下的 debian 和 target 目录：

```
./debian
 ├── build.sh            debian的一键构建脚本
 ├── compressed_files    存放预编译好的debian基础包
 ├── debian-build-service  构建debian基础包的依赖文件和live build配置文件
 ├── mk-base-debian.sh   选择debian版本，然后调用live build构建debian基础包
 ├── mk-image.sh         将debian根文件系统生成rootfs.ext4镜像
 ├── overlay             sunxi overlay层，存放sunxi通用文件(与具体芯片无关)
 ├── packages            存放预制的deb文件
 ├── readme.md
 └── tools               存放制作ext4格式镜像的工具和配置

./target
 └── {IC}            具体IC型号
   └── debian        debian文件夹，固定路径
     └── {BOARD}     具体方案名称
       └── overlay   overlay文件夹；此目录下的所有文件会在构建时复制到debian rootfs文件夹
```

3.2 配置

在 SDK 根目录执行./build.sh config，依次选择 linux，debian，内核版本，芯片型号，方案名等，最后选择 debian rootfs 预编译好的压缩包。

以下截取部分内容：

```
========ACTION List: mk_config ;========
options :
All available platform:
  0. android
  1. linux
Choice [linux]: 1
All available linux_dev:
  0. bsp
  1. dragonboard
  2. buildroot
  3. debian
Choice [debian]: 3

...

All available rootfs files:
  0. linaro-bullseye-gnome-arm64.tar.gz
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
  1. linaro-bullseye-lite-arm64.tar.gz
  2. linaro-bullseye-lxde-arm64.tar.gz
  3. linaro-bullseye-xfce-arm64.tar.gz
Choice [linaro-bullseye-xfce-arm64.tar.gz]: 3
```

3.3 构建

在 SDK 根目录执行./build.sh，即可开始构建。编译 boot0，uboot，内核等镜像。

最后编译 debian 根文件系统，在 out/{IC}/{BOARD}/目录生成 rootfs.ext4 镜像。

编译完后执行打命令./build.sh pack，生成固件烧录即可。

3.4 自定义 debian 基础包

./debian/compressed_files 目录存放的压缩文件是编译好的 debian 基础包；若需要自定义，参考debian 基础包编译章节

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 debian 开发

4.1 配置处理

4.1.1 配置流程

在 SDK 根目录执行./build.sh config，进行交互式配置。流程如下：

1、执行./build/mkcmd.sh 脚本：mk_config 函数 -> select_debian_rootfs 函数

2、进入./debian 目录，运行./build.sh config，根据 compressed_files 目录下的文件弹出选项。

3、选择压缩包，生成.config 文件。

debian/.config 文件内容参考：

```
DEBIAN_TAR_ROOTFS=linaro-bullseye-xfce-arm64.tar.gz
```

4.1.2 快速配置

对于自动化构建来说，交互式弹窗配置是不合理的。此处提供快速配置方法:./build.sh autoconfig，参数说明：

```
-o: 指定debian

-i: 芯片型号

-b: 方案板型名称

-n: emmc和nand介质为default, nor介质指定nor

-k: 内核版本，如：linux-5.15

-r: debian rootfs预编译压缩包名称，如：linaro-bullseye-xfce-arm64.tar.gz
```

则最终的快速配置命令为：

```
./build.sh autoconfig -o debian -i {IC} -b {BOARD} -n default -k 内核版本 -r 预编译rootfs压缩包名称

以t527为例：
./build.sh autoconfig -o debian -i t527 -b demo_linux_aiot -n default -k linux-5.15 -r linaro-bullseye-xfce-arm64.tar.gz
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.2 构建处理

4.2.1 构建流程

在 SDK 根目录执行./build.sh，属于整体构建，自动完成所有构建流程。编译了 boot0,uboot,kernel 等镜像，根据方案差异还会编译 rv,dsp 等镜像。

最后才编译 debian rootfs。此处主要说明 debian rootfs 的编译流程：

1、执行./build/mkcmd.sh 脚本：build_rootfs 函数 -> build_debian_rootfs 函数

2、进入./debian 目录，运行./build.sh；然后从.config 的 DEBIAN_TAR_ROOTFS 配置获取 rootfs 预编译压缩包名称

3、判断是否存在 SDK/out/{IC}/${BOARD}/rootfs_def 文件夹；若存在，则执行./mk-image.sh cover；不存在，则执行./mk-image.sh +rootfs 压缩包名称，如：./mk-image.sh linaro-bullseye-xfce-arm64.tar.gz；

4、执行 mk-image.sh 脚本，若传入参数不等于 cover，则先把 rootfs 压缩包解压到 SDK/out/{IC}/${BOARD}/rootfs_def 文件夹；

4.1、复制 debian/overlay，SDK/target/{IC}/debian/{BOARD}/overlay 文件夹下的文件，以及一些系统文件到 debian rootfs 根文件系统；

4.2、使用 make_ext4fs 工具将 SDK/out/{IC}/${BOARD}/rootfs_def 文件夹制作成 rootfs.ext4 镜像，生成镜像与 rootfs_def 放在同级目录；

4.2.2 快速构建

整体构建花费时间比较久，对于部分开发者来说，只是想重新生成 debian rootfs 镜像。有 2 种方法可以快速编译 debian rootfs。以下任选一种即可：

1、在 SDK 根目录执行: ./build.sh debian_rootfs

2、进入 debian 目录，然后执行: ./build.sh

说明

默认构建不会主动删除 SDK/out/{IC}/${BOARD}/rootfs_def 文件夹；若更换了 rootfs 预编译压缩包配置，或发现有 rootfs 有残留文件，可在 SDK 根目录执行：./build.sh debian_rootfs f。此命令会强行删除 rootfs_def 文件夹，然后再重新解压预编译压缩包，最后生成 rootfs.ext4 镜像。

4.3 debian 基础包编译

若./debian/compressed_files 目录的 debian 基础包已满足要求，可跳过此章节。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

需要自定义 debian 基础包的部分开发者需要按照此章节重新编译 debian 基础包。

4.3.1 环境搭建

推荐使用 ubuntu 20.04 的系统。低版本 ubuntu 编译有问题。

注意

需要 sudo 权限，以及网络要求可以连接外网。

安装命令如下：

```
sudo apt-get install binfmt-support qemu-user-static live-build
进入SDK/debian目录执行：
sudo dpkg -i debian-build-service/packages/*
sudo apt-get install -f
```

若遇到编译报错，根据报错内容继续安装相关的软件包。

4.3.2 编译

进入 debian 目录，执行: mk-base-debian.sh；选择 debian11，以及对应的桌面版本。

然后就会调用 live build 构建对应的 debian rootfs 根文件系统。

说明

live build 实际是一些脚本，位于/usr/lib/live/目录；它使用一个目录来方便的构建 debian rootfs。对应命令为 lb。
live build 官网: https://live-team.pages.debian.net/live-manual/

mk-base-debian.sh 脚本执行流程：

1、选择 debian11，选择桌面版本，芯片架构等

2、根据选择，进入对应目录 debian-build-service/debian 代号-桌面版本-芯片架构；如: debian-build-service/bullseye-xfce-arm64

3、运行 sudo lb clean –purge，删除一些之前编译过的中间文件

4、运行 lb config，具体参数参考 configure 文件

5、运行 sudo lb build，开始编译 debian rootfs 根文件系统

6、最终在 SDK/debian 目录生成压缩包：debian 代号-桌面版本-芯片架构-日期.tar.gz

最后将生成的压缩包替换 debian/compressed_files/目录对应的压缩包即可。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.3.3 软件源修改

若需要修改默认的软件源，则修改 debian/debian-build-service/debian 代号-桌面版本-芯片架构目录 configure 文件的以下参数:

```
--mirror-bootstrap "http://mirrors.ustc.edu.cn/debian" \
--mirror-chroot "http://mirrors.ustc.edu.cn/debian" \
--mirror-chroot-security "http://mirrors.ustc.edu.cn/debian-security" \
--mirror-binary "http://mirrors.ustc.edu.cn/debian" \
--mirror-binary-security "http://mirrors.ustc.edu.cn/debian-security" \
```

4.3.4 rootfs 软件包修改

若需要新增或者删减 debian rootfs 中支持的命令，则可以修改文件:

```
debian/debian-build-service/debian代号-桌面版本-芯片架构/customization/package-lists/linaro.list.chroot
```

新增命令，则在文件中添加即可；如新增 docker 命令，则在文件中添加一行：docker.io

删减命令，则在文件中直接删除即可。

最后重新编译 debian 基础包。

4.4 overlay 层叠加

开发者往往有自己的配置文件或编译好的二进制文件，需要安装到 rootfs 中。

按照优先级从低到高开始介绍。

层级 1: debian/overlay 目录；存放所有芯片通用的文件。

层级 2: target/{IC}/debian/common/overlay 目录；存放一个芯片通用的文件。

层级 3: target/{IC}/debian/{BOARD}/overlay 目录；存放一个芯片某个方案特有的文件。

debian/mk-image.sh 脚本中会依次复制 overlay 目录下的文件到 rootfs 中。编译完后，可在 SDK/out/{IC}/${BOARD}/rootfs_def 文件夹中查看是否有按预期的安装了文件。最后才打包烧录，在设备看效果。

说明

若 3 个层级的文件夹中都存在同样的文件，以层级 3 的为准。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。